AIStudio Tutorial

Version: Beta | Updated: 2026-07-12

Get the most out of AIStudio with four guided modules — from your first query, to two production-scale corpora, to your own documents.

> ***Prerequisites:*** *AIStudio must be installed and running. If you haven't done that yet, start with QUICKSTART.md.*
>
> *Sections marked Going deeper explain the **why** behind the **how**. Skip them on a first pass — the steps work without them. For the full mechanics behind a step, the **Annexes** carry the in-the-weeds reality the modules deliberately leave out.*
>
> ***Radical transparency.*** *AIStudio is built so you never have to take its word for anything. Every claim in every answer carries a clickable citation to the exact source and page — verify it in two clicks. It also doesn't hide that the answer depends on the **model**: ask the same question of two models and the emphasis shifts (§2.5 shows this side by side), and the citations are how you tell which framing the documents actually support. And where the happy path ends — the pre-verified data and default settings Modules 2–3 walk — the body says so and points you to the Annex that carries the messier truth. The annexes are the payoff, not an apology. The tool's job is not to sound authoritative; it is to make its reasoning checkable.*

Module 1 — Quick Tour

Goal: Run your first query, understand citations, explore the interface. ~15 minutes.

1.1 Your First Query

AIStudio ships with a **demo** corpus — original documents spanning enterprise architecture, IT strategy, financial-services technology, and agentic AI. It's already indexed and ready.

Open the **Terminal** app first (press **⌘ Space**, type `Terminal`, press **Enter**), then start AIStudio. The `ais_*` commands work from anywhere once installed, so you don't need to `cd` into the

repo for `ais_start` — but if a command ever reports it can't find the project, run it from inside `~/Developer/AIStudio`.

Open AIStudio:

```
ais_start
```

> ***If `ais_start` doesn't come up:*** *run `./ais_install` (rebuilds the venv and re-registers the `ais_*` commands), then `ais_start` again. Still stuck? See QUICKSTART Step 8 (troubleshooting).*

When AIStudio opens, the **left sidebar** is your control panel. The **CORPUS** section near the top is where you choose which document set to query — **demo** is pre-selected. **New**, **Delete**, and **Rename** do what they say; the **Edit** button (corpus settings) we cover in §4.6. Below them, two buttons manage a corpus's files: **Add** ingests a new file into the selected corpus, and **Inspect** lists the corpus's files so you can remove or re-index them.

We'll build a short **three-question thread** across §1.1–§1.3 that walks the whole loop — grounding, then conversation memory, then multi-source citations. Start with the opener:

- *"How do you design an IT organization around architectural principles?"* ← **start here**

You'll get a grounded answer from the FS Journal *Strategy and Architecture* paper — a systematic analysis bridging business vision, organizational constructs, processes, and enabling technologies — with each claim cited to a specific page and a clickable **Source Dive** back to it. Your first look at a grounded, attributable answer.

> ***Tip — recall a question.*** *Press ↑ / ↓ in the question box to scroll back through questions you've already asked this session, edit, and re-send — no retyping.*

1.2 Understanding Citations

Every answer carries citations — `[1]`, `[2]` — and a **References** panel showing exactly which document and page each claim came from. **The inline `[N]` markers are clickable**: click one — or the **Open** ↗ on its References entry — to open the source in the **Source Dive** panel at the cited page (PDFs); other file types open in a new tab. This is AIStudio's core promise: answers grounded in your documents, with a path back to the source.

> ***A note on citation reliability across machines.*** *Citation behavior is not perfectly uniform across hardware. In our own testing the same query, corpus, and model produced clean inline citations on higher-memory Macs (e.g. 64–128 GB unified memory) while a memory-*

> *constrained machine occasionally returned a well-grounded answer that omitted the* `[N]` *markers — same prompt, same model, different box. We've traced part of this to how much context the answer is built from (large retrieval sets crowd a smaller model's instruction-following) and are actively investigating the role of available memory in particular. The practical takeaway for now: if you see answers that are correct but uncited on a lower-memory machine, lower* ***Top K*** *(fewer, tighter sources) and prefer the smaller model for your RAM tier — both reliably restore citations. This is an open area of work, tracked in the project notes; the grounding itself (the retrieved passages) is unaffected — only whether the model renders the* `[N]` *marker.*

1.3 Follow-up Questions

AIStudio keeps conversation context across a session, so you can build on the previous answer with pronouns — it remembers what *that* refers to. Continue the thread from §1.1:

- *"How does that differ from just drawing org charts?"* — *that* = designing an IT organization around architectural principles, from your first question; the model resolves the pronoun across turns, and the answer now draws on **several documents at once** — watch the **References** panel fill with multiple sources (five, here), each claim carrying its own clickable `[N]` and **Source Dive**.
- *"What domains can the notion of "Product Management" apply to in an IT organization?"* — a fresh, broadening question; grounded in the *Architecture Concepts* paper (solution engines, user interface, engineering, integration operations, and more), each domain cited to its page.

1.4 Tuning Your Query

In the **Query Settings** section of the sidebar — just below the CORPUS area — are the knobs that shape how AIStudio retrieves and answers. Let's walk each one.

Top K is how many document chunks AIStudio retrieves to answer from: - The demo ships with **Top K = 10** (you'll see 10 in the field) — it includes small documents (the QFD paper is 34 chunks) that get outcompeted at lower K and surface reliably at 10. *(A corpus that stores no default of its own falls back to the system default of 5.)* - **Use 10 for SEC 10-K and ESEF** — these are the large financial-filing corpora you'll build in Modules 2 and 3 (annual reports from many firms). There you'll ask cross-firm *comparison* questions, which need chunks from several filings at once. AIStudio also raises Top K automatically when a query names several firms (~2 more chunks per firm, ceiling K=20). - **For your own corpora**, this is the main retrieval knob to tune: raise it (**15–20**) for broad or synthesis questions, or sparse corpora where the relevant context is scattered; lower it (**~5**) for precise single-fact lookups, to cut noise and latency. Too-high a K dilutes the prompt and can *worsen* a small model's answer.

Temperature controls creativity (0.1–0.3 precise, 0.5–0.7 more synthesis; default 0.3 suits document Q&A).

Retrieval Mix is a slider that runs **left → right = literal → conceptual**. Drag **left** for literal matching — exact terms, tickers, defined phrases, names; drag **right** for conceptual matching — themes and ideas where the wording varies; the **middle (default) blends both**. (Under the hood the slider blends keyword/BM25 and semantic/vector retrieval; the panel shows it in plain literal↔conceptual terms.) Lean **literal** for exact terms, tickers, and acronyms (CET1, firm names); lean **conceptual** for paraphrased or thematic questions — see **Annex 5 (BM25)** for the mechanics.

Relevance Threshold is a relevance floor — retrieved chunks scoring below it are dropped before the model sees them, which suppresses weak, off-topic context. The default is **0.5**, and **each corpus can set its own** (demo ships at 0.3) — you change it per corpus in the corpus **Edit** panel (§4.6). **When to lower it:** the embedding model (`nomic-embed-text`) often scores genuinely-relevant chunks below 0.5, so if good answers come back hedged or empty, drop the floor toward **~0.2–0.3** and re-ask.

Timeout is how long AIStudio waits for the model to answer before giving up — the default is **300 seconds**, deliberately generous because large models are slow (a 70B answer can take ~140–170s, and cutting it off mid-generation wastes the whole run). Like Relevance Threshold, **each corpus can set its own** (in the **Edit** panel), and you can override it per query in the left panel. **When to change it:** raise it only if you switch to a very large model and see timeouts; there's rarely a reason to lower it. Most users never touch this one — it exists so a slow-but-valid answer on a heavy model isn't cut off.

All three of these — Top K, Relevance Threshold, Timeout — follow the same resolution: the left-panel value wins for that query; leave it as-is (or blank) and the corpus's saved default applies; if the corpus stored none, the system default takes over. Tune once per corpus in **Edit**, or ad-hoc per query.

1.5 The Help Corpus — AIStudio Answering About Itself

Switch to the **help** corpus and ask:

- *"How do I re-ingest a corpus?"*
- *"What embedding model does AIStudio use?"*

Same RAG pipeline, pointed at AIStudio's own docs. Try it before reaching for the manual.

Module 2 — At Scale: The SEC 10-K Corpus

Goal: Build a large corpus from scratch, give it the reference data it needs to retrieve well, and query it. ~20 minutes (6 min ingest).

This module downloads a portfolio of SEC 10-K annual filings, ingests them, and queries them. It also introduces the part of AIStudio that makes retrieval at this scale work: **knowledge bases**. The corpus ships with a benchmark question set; running it and — more importantly — reading it correctly is **Module 5**.

2.1 Download Filings

```
ais_download_sec_10k
```

This reads the corpus's **membership list** (a manifest) and pulls each firm's **most recent** annual 10-K filing from SEC EDGAR into the corpus — **20 filings** across the portfolio. Allow a few minutes. (`--latest 5` selects the five newest filings per firm; the bare `ais_download_sec_10k` defaults to the single most recent.)

EDGAR serves filings by **CIK** — the *Central Index Key*, the unique number the SEC assigns to every entity that files with it (look one up at the SEC's CIK lookup). So preparing this corpus came down to building a small **manifest** — one row per firm, pairing a **Label** (the name AIStudio uses for the firm throughout the system) with two identifiers: its **CIK**, the key EDGAR needs to return the filings, and a verified **LEI** (the 20-character global Legal Entity Identifier), which §2.2 puts to work. In short: a manifest of CIKs and LEIs. Its full structure is in **Annex 1**.

> ***What is SEC EDGAR?*** *The SEC requires public companies to file annual reports (10-K); EDGAR is the public database where they live. AIStudio's downloader handles the access protocol — you get the files, it handles the plumbing.*
>
> ***Firms included:*** *Goldman Sachs, JPMorgan Chase, Morgan Stanley, BlackRock, BNY Mellon, Citigroup, and others — bulge-bracket banks, asset managers, exchanges, custody banks.*

Going deeper — Why these filings need more than raw text

(skippable)

In Module 1 you queried the **demo** corpus — and if you select it and click **Inspect**, you'll see it's nothing but plain PDFs dropped straight in (Module 4 walks you through building a corpus that way from your own PDF or text files). General documents like these are written *for* a reader — topic, dates, subject are all in plain prose.

Filings are different: hundreds of pages of tables, footnotes, and boilerplate. Even *"what year is this filing for?"* is hard to answer reliably from raw text — you'd need a brittle heuristic that breaks on the next corpus. We want something corpus-independent.

The companies already solved this at the source. Filings aren't loose text — they're filed in a standardized, machine-readable format: **iXBRL (inline XBRL)**, an HTML document with embedded tags that codify the key facts, including **which entity** the filing covers and **what period**. So the first thing AIStudio does at ingestion is read those tags directly — pulling the company name and fiscal year with no reliance on the filename.

2.2 Recognizing Unique Company Names

AIStudio reads two iXBRL tags from each filing:

- `dei:EntityRegistrantName` — the company the filing is about
- `dei:DocumentFiscalYearFocus` — the SEC-mandated fiscal year

That anchors *what this document is* — but it isn't enough. The same firm appears as *"JPMorgan"*, *"JPMorgan Chase"*, *"JPM"*, or a ticker, while the filing's registered name is `JPMORGAN CHASE & CO.` Retrieval has to know these are the **same entity**.

AIStudio handles this with an **entity knowledge base**: a mapping from each firm's canonical legal name to its natural variants. The canonical name and its aliases are pulled from **GLEIF** (the Global Legal Entity Identifier Foundation — the authoritative open registry of legal entities) **by LEI**, then enriched with short names and tickers.

The crucial design point is *how* a firm is looked up. You might expect a search by company name — but legal names are ambiguous: a multinational has dozens of subsidiaries, depositary receipts, and namesakes, and a name search confidently returns the *wrong* entity often enough to corrupt a corpus silently. So AIStudio resolves each firm by its **LEI** — the 20-character Legal Entity Identifier that names exactly one entity worldwide. **The LEI is the input, not a guess:** the corpus ships with a verified LEI for every firm.

Build the entity KB:

```
ais_import_entity_kb --corpus sec_10k --apply
```

This scans the downloaded filings, resolves each firm by its verified LEI against GLEIF to pull the canonical name and aliases, and writes the **entity knowledge base** — keyed by the name each filing reports for itself, carrying that firm's canonical name, LEI, and aliases. (Its exact structure and where it's stored are in **Annex 1**.)

> *For the shipping portfolio every firm resolves cleanly on its verified LEI, so this is one command. When you build entity support for your **own** corpus (Module 4), you supply the*

> *LEIs — and if a firm doesn't resolve, the command pauses and hands you a worksheet to confirm it. **Annex 1** is the full story: what goes wrong when a name search guesses, and the one-time review gate that catches it.*

Going deeper — How a chunk knows which company it's about

(skippable)

At ingestion AIStudio splits each filing into many small **chunks** and embeds each one. A query is embedded the same way, and retrieval matches question to chunks. The problem at scale: a chunk of dense financial text, lifted out of a 200-page filing, often carries no clue which firm it came from — a paragraph on capital ratios reads almost identically across twenty-five banks.

AIStudio fixes this with **chunk enrichment**: at ingestion, every chunk is prefixed with a small label carrying the company name, its aliases, and the filing year — drawn from the entity KB and the iXBRL tags:

```
[Document: THE GOLDMAN SACHS GROUP, INC. | Goldman Sachs | GS FY2025] <chunk text…>
```

Now identity travels *with* the text. When your question names a firm, retrieval can restrict to that firm's chunks — and because the label carries aliases, it doesn't matter whether you typed "Goldman", "Goldman Sachs", or "GS".

2.3 Industry Terms — the Glossary

Company names aren't the only vocabulary gap. Ask about *"CET1"* and the passage may only ever say *"Common Equity Tier 1 capital ratio"*, or vice versa.

AIStudio bridges this with a second knowledge base — a **glossary** of domain terms and expansions, sourced for the bank corpora from the **BIS Basel** framework. It ships ready-built. (Its structure and storage are in **Annex 2**.)

Build (or refresh) it with:

```
ais_import_glossary_kb --source bis_basel
```

Unlike the entity KB, the glossary is **corpus-wide** — "CET1" means the same thing everywhere — and it needs no review gate (see Annex 2).

> ***Before you ingest:** the entity KB (§2.2) must exist at ingestion time — chunk enrichment depends on it. The glossary needs nothing built at ingestion; it's read later, at query time.*

Going deeper — When the glossary is applied (query time, not ingestion)

(skippable)

An asymmetry worth understanding. The **entity** label is baked into each chunk *at ingestion* — it becomes part of the stored text. The **glossary** works the other way: chunks are **not** rewritten; the mapping is applied to **your question, at query time**. Ask about *"CET1"* and AIStudio widens the search to include *"Common Equity Tier 1 capital ratio …"* before retrieving — so an acronym query finds a spelled-out passage and vice versa, with the stored chunks left untouched. This is the spine of the two reference layers: entity = build-time + query-filter, glossary = query-time expansion.

2.4 Ingest the Corpus

With the entity KB built and AIStudio running:

```
ais_ingest_sec_10k
```

~6 minutes on an M4 MacBook Pro. This is where chunk enrichment happens — as each filing is chunked, AIStudio reads its iXBRL entity and year and prefixes every chunk with the `[Document: …]` label. The summary reports how many files were augmented and which entities it recognized, and closes with a `· File Ingested:` roster of every file written this run.

> *Leave the terminal open; ingestion is CPU- and memory-intensive.*

2.5 Query at Scale

Switch to the **sec_10k** corpus (the breadcrumb at the top should now read `AIStudio · sec_10k · …` — that's how you confirm which document set the next answer draws on; changing the *model* does **not** change the corpus, they're separate dropdowns).

First, a word on **model choice**. AIStudio runs its models locally through Ollama, and you can add as many as you like. For the easy Module 1 questions, small and large models are about equal. That changes here: SEC filings are dense, and questions that span several firms or turn on a precise disclosure reward a model with stronger synthesis and tighter citation discipline. So for this module, **select `gemma3:27b` in the model dropdown** — the synthesis model AIStudio is tuned around. If you don't have it yet, pull it once through Ollama — `ollama pull gemma3:27b` (it's a one-time ~17 GB download; QUICKSTART Step 4 and HOWTO cover model management in full).

Start with one firm

A question that **names a firm** is the easy case — the name acts as a natural anchor, so retrieval lands on that firm's filing and a modest **Top K of 10** is plenty:

- *"What are JPMorgan's primary cybersecurity risks?"*

You'll get a focused answer — unauthorized access, data manipulation, ransomware, multi-source attacks — citing `JPMorgan_Chase_10K_2026-02-13.htm` and nothing else. That single-source citation is the point: at scale, with 20 firms in the index, retrieval still isolated to the firm you asked about. Try another:

- *"How does Goldman Sachs describe its approach to AI and machine-learning risk?"*

Then ask across firms — and watch what happens

Now ask a question that **doesn't name any firm** but expects several:

- *"Which of these firms disclose a dedicated climate-risk or sustainability governance committee, and how do they differ?"*

At **Top K = 10**, this question may come back thin or empty. That isn't a bug — it's the most important lesson in this module. **Top K is a global budget**: it retrieves the 10 best-matching chunks *in total*, across the whole corpus. When you name a firm, those 10 chunks concentrate on that firm. When you *don't*, they scatter — and with 20 firms competing, 10 chunks isn't enough to cover several of them, so the answer starves.

Raise Top K to 20 and re-ask. Now it answers — but read it carefully: it likely surfaces only **two or three** firms, not all 20. Raising K helped, but a global budget still clusters on whichever firms have the densest matching language; the rest are silently left out. The answer is *correct for the firms it found* and should be read as "among the firms retrieved…", not "these are the only firms that qualify."

The fix: name the firms you want compared

If you know which firms you care about, **name them** — that gives retrieval an anchor per firm:

- *"Compare how JPMorgan, Citigroup, and Goldman disclose climate governance."*

At Top K = 20 this now surfaces **all three**, each with its own distinct framework (JPMorgan's Climate, Nature and Social Risk Management function; Citigroup's Climate Risk Management Framework; Goldman's Firmwide Enterprise Risk Committee), and it even flags where a firm's disclosure is less explicit. Naming the entities turned an open question that starved into a precise one that delivered.

> ***The takeaway — match Top K to the question.*** *Named, single-firm or few-firm questions work at* ***K=10****. Open "which firms…" questions need* ***K=20+*** *and will still under-cover a large corpus — so for a real comparison,* ***name the firms****. For your own corpora later, this is the main retrieval knob to tune (see §1.4 and the annexes): raise K for breadth, lower it for precision, and anchor cross-entity questions with explicit names. AIStudio is exploring query understanding so that an open "which firms…" question can fan out across entities automatically — for now, naming them is the reliable path. The annexes cover why this happens (entity coverage vs. lexical density) and the directions being explored.*
>
> ***See what a corpus covers — and what it doesn't.*** *Ask "which companies are covered?" and AIStudio lists the corpus's firms straight from its own metadata — a deterministic answer, not a model guess. And if you ask about a firm the corpus doesn't hold, you get an honest* ***"No information on X was found in this corpus"*** *that names what is covered — never a confident answer stitched together from the wrong company. For a product whose whole thesis is grounded, cited answers, confidently citing the wrong firm would be the worst failure available, so AIStudio refuses it by design.*
>
> ***Switching models keeps your conversation.*** *Once you've added more than one model, re-ask the same question on each from the dropdown and compare. AIStudio prints a brief "Changed to <model>" line, and your ↑ / ↓ question history works across the switch, so you can recall a question and send it to the new model without retyping.*
>
> ***Worked example — the same question, two models.*** *Ask "How does Goldman Sachs describe its approach to AI and machine-learning risk?" on* ***gemma3:27b****, then switch to a smaller model (e.g.* ***mistral:7b****) and press ↑ then Enter to re-ask. Both answers are grounded and cite real Goldman filings — but they differ. The larger model is more exhaustive; the smaller is tighter and faster, covering the same core in fewer words. Neither is "wrong." The model sets depth and emphasis; the citations are how you confirm which framing the filings actually support — radical transparency made concrete: read the sources, don't take the wording on faith.*
>
> ***When the answer is a number from a table, read it twice.*** *A cross-firm numeric query — "Compare the CET1 ratios for JPMorgan and Citigroup" — is where AIStudio is most likely to return a confident wrong figure: financial ratios live in multi-year, multi-column tables, and a chunk can sever a cell from the column header (the year) that gives it meaning.* ***Annex 4*** *is the full worked case — one of those two firms comes back exactly right and the other a full year off, and the contrast is the whole lesson.*

2.6 Add a Firm on Demand

You don't have to rebuild the whole corpus to add one company. The flow is **download** → **(refresh entity KB)** → **selective re-ingest** → **query**, and it leaves the other filings untouched.

1. Download just the new firm (by ticker — download stays a manual step):

```
ais_download_sec_10k --tkr BLK --latest
```

That fetches BlackRock's most recent 10-K into the corpus's `uploads/`. One firm, one command.

> ***Going deeper* — *corporate events change a firm's filing identity.*** *A ticker resolves to a company as it files today — but a firm's identity in EDGAR isn't fixed.* ***Corporate events*** *— mergers, spin-offs, name or ticker changes, redomiciles, holding-company reorganizations — can split one business's filings across more than one identifier over time.*
>
> *BlackRock is a clean case. On October 1, 2024 it completed a holding-company reorganization: the predecessor company (EDGAR CIK `0001364742`) was replaced by a new "BlackRock, Inc." (CIK `0002012383`), each old share converting one-for-one into the new. The business is continuous, but EDGAR now indexes it under* ***two*** *CIKs — the new one carries FY2024 onward, the predecessor the earlier years — and `BLK` only resolves to the new one. So to build a full multi-year history you fetch each era explicitly:*
>
> ```*bash*
>
> *# FY2024–FY2025 — current company (the ticker resolves here)*
>
> *ais_download_sec_10k --tkr BLK --latest 2*
>
> *# FY2021–FY2023 — predecessor, by its CIK, labeled to the same firm*
>
> *ais_download_sec_10k --cik 0001364742 --force_name "BlackRock, Inc." --latest 3* ```

2. (Optional) Refresh the entity KB so the new firm gets its aliases. The other firms carry ticker/short-name anchors (e.g. `Citi`, `C`) built in §2.2. To give the new firm the same — so a *ticker-only* query like "BLK" anchors as well as the full name — refresh the KB before re-ingesting:

```
ais_import_entity_kb --corpus sec_10k --apply
```

For a firm you'll query **by name** (as we do below), you can skip this — name-anchoring works without it. (See Annex 1 §A1.5 for how the KB resolves a firm's aliases.)

3. Re-ingest to make the new firm queryable. Downloading only puts the filing on disk — the firm won't appear in answers until you ingest it. (Ask about BlackRock right after the download and AIStudio correctly says it has nothing: the file exists but isn't indexed yet.) Re-ingest, naming just the new firm so the other filings are left untouched:

```
ais_ingest_sec_10k --files BlackRock
```

`ais_ingest_sec_10k` is incremental — it skips files already indexed — so `--files BlackRock` adds only the new BlackRock filings in one pass. `--files` takes one or more comma-separated **patterns**, OR-matched against the filename: each is a literal substring, or a regex if it contains regex metacharacters (`*+?[]()|^${}\`). So `--files BlackRock` matches every BlackRock filing on disk and ingests them in one pass; `--files 'JPM.*2025,Citi'` matches either. Everything unmatched is left untouched, and the run ends with a `· File Ingested:` roster of exactly what landed.

In the predecessor download above we passed `--force_name "BlackRock"` so those filings carry the same firm label as the current company — otherwise they'd file under the predecessor's own registrant name and read as a separate firm. (For how AIStudio maps a company across its identifiers — CIK, ticker, LEI, and the registrant name in the filing itself — see **Annex 1 §A1.5**.)

4. Query it. Switching back to AIStudio in the browser, select **sec_10k** and ask a question only BlackRock can answer — that's what surfaces its filing on its own merits:

> *What is BlackRock's Aladdin platform, and what role does it play in the firm's business?*
>
> ***One firm, many identifiers.*** *A company appears under several codes across naming standards — CIK, ticker, LEI, the registrant name in the filing itself — and AIStudio records them in an open schema where a human-verified value always wins over a machine's name-search guess. Again, Annex 1 §A1.5 walks one such one-edit correction — promoting BlackRock's LEI to its verified value — as the worked case.*

2.7 The Full Picture — Multi-Year

Everything so far used **the latest fiscal year only** — with one exception: the prior-year BlackRock filings you pulled in §2.6's *Going deeper* box. That single-year default — one filing per firm, the most recent — is chosen so the first ingest is fast (you saw 20 filings ingest in a few

minutes). But a single year can't answer the questions a 10-K corpus is really *for*: how things **change**. Capital ratios tighten or loosen, risk language shifts, a new regulation appears in the 2026 filing that wasn't in the 2022 one. To see that, you need several years per firm.

The downloader takes parameters for exactly this. Your most-recent-year run was the default — equivalent to `--latest 1`. Two flags widen the scope:

```
--latest N          Download the N most-recent 10-K filings per firm
--years YYYY ...    Download specific fiscal years (space-separated, e.g. --years 2022 2023
```

> ***Watch the `--years` meaning.*** *`--years` takes **explicit fiscal years**, not a count. To get "the five most recent," use `--latest 5` — not `--years 5` (a bare small number is rejected with a hint, since no firm filed in year 5 AD).*

BlackRock already has all five years (you pulled them in §2.6). Adding the four prior years to the other **20 firms** means **20 × 4 = 80 new filings**, bringing the corpus to **105 filings (= 21 firms × 5 years)**.

Expanding the corpus is **three steps** — you re-run the same sequence you did for one year, and each stage is incremental, touching only the new filings:

```
ais_download_sec_10k --latest 5              # 1. fetch the 4 prior years you don't have yet
ais_import_entity_kb --corpus sec_10k --apply # 2. refresh the entity KB so the new filings
ais_ingest_sec_10k                           # 3. ingest — incremental, only the 80 new file
```

> ***Where the time goes.*** *The download is quick — the filings are a few hundred MB and EDGAR is fast. **The cost is the indexing.** Each filing has to be chunked, entity- and temporal-prefixed, and embedded — the per-file rate you saw in §2.4. So the 80 new files are the bulk of the wait, almost all of it in step 3 (ingest), not step 1 (download). The ingest banner's time estimate scales to the number of files actually queued.*

What this unlocks

With multiple years indexed, you can ask **temporal** and **cross-firm-over-time** questions — the kind a single-year corpus simply cannot answer:

- *How has climate-risk and Net-Zero / transition-risk disclosure evolved across these filings since 2022 — and which firm has the most detailed framework?*
- *How have the major banks described their digital-banking strategy and technology-investment priorities over the last five years — what themes emerge across firms?*

- *What new regulatory, capital, or compliance burdens appear in the 2025–2026 filings that weren't in 2022 — and which do firms agree are hardest?*

Each asks the model to retrieve across **both** the firm axis and the year axis, then synthesize a trend.

A fair warning — these are the hard ones

Multi-year, multi-firm questions are **among the most difficult retrieval-and-synthesis tasks in this whole tutorial**, and it's worth understanding why before you judge the answers:

- **The intent is implicit.** "How has X evolved?" names no year, firm, or metric — the system has to infer you want a comparison *across* the years it holds. Reading that intent is its own research problem: query understanding — inferring what a question is really asking before retrieving against it. (You saw a smaller version of this in §2.5: an open "which firms…" question starves where naming the firms succeeds.)
- **Retrieval has to span partitions.** A good answer needs chunks from *several* firms *and* several years; a global top-K can over-sample one verbose firm or one year and miss the rest — the entity-coverage-vs-lexical-density effect from §2.5, now multiplied by the year axis.
- **The needles are often in tables.** Year-over-year numbers live in financial tables, the hardest content to extract and align (see the CET1 note in §2.5 and Annex 4).

AIStudio's current approach to these — entity-anaphora normalization, temporal-context injection at ingest (the `[Document: <entity> FY<year>]` prefix), and the directions being explored for query understanding — is laid out in the **annexes**. Treat the multi-year answers as a window into an open problem, not a solved one: read them alongside Module 5 (benchmarking), which is precisely about reading these results *correctly* rather than at face value.

Module 3 — A Second Corpus: European Banks (ESEF)

Goal: Build a second production corpus that mirrors Module 2 — same four steps, a different regulator and access key. ~40 minutes.

The SEC corpus is US filers retrieved from EDGAR by **CIK**. European banks file under the EU's **ESEF** (European Single Electronic Format) mandate, retrieved from **filings.xbrl.org** by **LEI**. The shape of the work is identical — download → entities → glossary → ingest — which is the point: the corpus machinery is source-independent, only the access key and the source endpoint change.

3.1 Download the Filings

```
ais_download_esef
```

This reads the corpus's scope file (`esef_banks_full_scope.yaml`) — one row per bank, keyed by LEI — and queries filings.xbrl.org by LEI for each bank's ESEF annual reports, into `data/corpora/esef_banks/uploads/` .

> ***Filing years may differ across firms.*** *`ais_download_esef` fetches the latest available filing for each firm — but "latest available" is not always the same fiscal year for every firm. Some banks file months after the calendar year end; others have not yet published. In a corpus built in mid-2026, BBVA and SEB were at FY2024 while the other EN-primary firms were at FY2025. Check the download summary and each firm's year in the `---Downloading` output before running cross-firm year-over-year queries.*
>
> ***Why LEI here and CIK for SEC?*** *Each regulator exposes a different access key — EDGAR indexes by CIK, filings.xbrl.org filters by LEI. AIStudio keeps the source specifics (endpoint, access key) as data, not code, so adding a regulator is a configuration change, not a rewrite. The **LEI is still the identity** in both — see Annex 1.*

3.2 Recognizing Unique Company Names

```
ais_import_entity_kb --corpus esef_banks --apply
```

Same command as SEC, pointed at the European corpus: it scans the ESEF filings, resolves each bank by its verified LEI, and writes `data/knowledge_sources/gleif/esef_banks_full_entities.yaml` .

> ***This is where Europe gets interesting.*** *A name search for these banks fails far more often than for US filers — subsidiaries, depositary receipts, and US-jurisdiction defaults mismatch firms like Société Générale, Crédit Agricole, and BBVA. That's exactly why the LEI is the input and not a guess. On the shipping portfolio the verified LEIs make this one clean command; the failure modes — and the one-time worksheet that catches them — are **Annex 1**, and the deeper "why non-English filings are harder" is **Annex 3**.*

3.3 Industry Terms — the Glossary

```
ais_import_glossary_kb --source bis_basel
```

This is the **same** glossary you built in §2.3, not a new one. Basel vocabulary (CET1, leverage ratio, NSFR, RWA…) is identical for US and European banks, so it's stored **corpus-agnostically** (`any_corpus`) and built once. If you ran Module 2 it already exists — re-running is a harmless idempotent refresh, here only so this module stands on its own. Its job is unchanged from §2.3 *Going deeper*: at **query time** it expands an acronym to its spelled-out form and back, so *"CET1"* and *"Common Equity Tier 1"* retrieve the same chunks. There is nothing ESEF-specific to build.

3.4 Ingest the Corpus

```
ais_ingest_esef
```

Same enrichment as SEC: each chunk is prefixed `[Document: <entity> | <aliases> FY<year>]` from the entity KB and the iXBRL tags.

3.5 Query at Scale

Switch to the **esef_banks** corpus and try these three, in order:

- *"How does HSBC describe its CET1 capital position?"* — an English filer. The glossary expands *CET1 → Common Equity Tier 1*, and the answer comes back specific and cited.
- *"What do European banks say about climate-related financial risk?"* — no firm is named, so this tests **breadth**: expect several banks cited side by side.
- *"How does BNP Paribas describe its CET1 capital position?"* — **the language case.** BNP files in **French**, so an English query is reaching across languages.

That third question is where **model size** earns its keep. Ask it on a small model and the answer is a vague, *uncited* hedge — it knows the question is about BNP but can't pull the specific figure across the language gap. Switch the model to `gemma3:27b` and re-ask: it bridges *fonds propres de base de catégorie 1* → CET1, retrieves the specific passage, and grounds it with a citation. **The retrieval mechanics did not change — only the model did.** That is the whole point of **Annex 3**: retrieval quality is not language-neutral, and on non-English filings a larger model is the lever that recovers the specific fact a smaller one misses.

Going deeper — When the portfolio isn't in English

(skippable)

A US-only corpus hides a problem that a European one surfaces immediately: retrieval quality is not language-neutral. A question in English against a filing that discusses *fonds propres de base de catégorie 1* instead of *Common Equity Tier 1* retrieves worse, and a firm whose filing

language is mislabeled retrieves worse still. The glossary helps with terminology, but it isn't the whole story. **Annex 3** is the full account of what degrades when the portfolio leaves English.

Module 4 — Bring Your Own Corpus

Goal: Ingest your own documents and query them. ~15 minutes + ingest.

Modules 2 and 3 used the operator seed machinery that ships the SEC and European corpora. Your own corpus uses the **UI** — you build directly what those corpora arrive as at runtime.

4.1 Create a New Corpus

In the browser, in the left **CORPUS** panel, click **New**, name it (e.g. `my_docs`), **Create**. This makes `data/corpora/my_docs/` with an `uploads/` folder.

4.2 Upload Your Documents

With the corpus selected, click **Add** and choose files. Supported: PDF (page-aware), DOCX, PPTX, XLSX, Markdown, HTML. AIStudio ingests automatically after upload and shows progress in the chat area.

4.3 Query Your Corpus

Select your corpus and ask. Same interface, your content.

4.4 Optional — Entity support for your own corpus

If your documents are entity-centric (filings, reports about specific firms), you can give your corpus the same entity KB the shipped corpora use — but **you supply the verified LEIs**, because there's no pre-built seed for a user corpus. That is precisely the workflow **Annex 1** documents end to end: scan, review the worksheet, fill the LEI where the guess is wrong, apply.

4.5 Optional — Write Your Own Benchmark Questions

Create `benchmarks/<your-corpus>/<your-corpus>_questions.yaml`:

```
- topic: Topic Name
  questions:
    - id: unique_id
      question: The exact question sent to AIStudio.
      keywords: [term1, term2, term3]   # all must appear in the answer to pass
      notes: What a correct answer looks like.
```

Then `ais_bench --corpus your-corpus-name`. What the pass/fail checks really mean is **Module 5**.

4.6 Optional — Add Corpus Guidance

Use the **Edit Corpus** modal to set description, summary, and search guidance (routing hints that tell the model which document to consult for which question) — written straight to the runtime `<your-corpus>_corpus_metadata.yaml`. No YAML editing required.

Module 5 — Benchmarking

Goal: measure a corpus scientifically — and learn to read the score correctly.

Benchmarking is how you find out what actually makes a difference — model choice, retrieval settings, entity handling, question shape. Without a measurement you are guessing. That is why we built a benchmark tool, and why it has carried this whole project: it lets us measure progress and find what works run after run, **scientifically** rather than by impression. `ais_bench` is easy to run and easy to *misread*, though — the naive score lies in a specific, dangerous direction, and the audit discipline below exists to catch that lie.

5.1 — A benchmark run is a point with coordinates in a multi-dimensional space, not just "the questions"

A run isn't simply *ask the questions*. It evaluates one point in a space of choices — **which questions × which firms (the scope) × the retrieval settings × how entities are handled × which model** — scored by a grading method. Reproducibility means writing that coordinate down: the question file and the scope file *are* the experiment record, and the run flags are its conditions. That is why the tests live in files, not in code.

The settings that move the result, and why each sits where it does:

- **Top K = 10** for multi-firm financial corpora — fewer slots drop firms from a comparison; 5 is fine for small single-topic corpora.
- **Retrieval Mix (α) = 0.5.** Pure conceptual retrieval loses the *named firm*: the query embedding is dominated by the shared concept (every bank discusses "CET1"), so the firm names barely move it and retrieval latches onto whoever wrote most densely about the topic. Blending in literal matching restores the names. 0.5 is the locked default.
- **Relevance Threshold (min_score).** A relevance floor that drops weak chunks — but set too high it starves genuinely relevant ones (the embedding model under-scores them), so ~0.2 is the working value for these corpora.

- **Entity handling — isolation, not expansion.** The central lesson: recognizing a firm and *appending* its name to the query does **not** stop the wrong firms' chunks from being retrieved. Only a retrieval-time **filter** keyed to the recognized firm isolates correctly — and AIStudio builds that filter automatically. Expansion decorates the query; the filter excludes the contaminants. Conflating the two once produced answers that cited the wrong companies while scoring 100% mechanically.
- **Model.** When retrieval is the bottleneck, model size barely moves the score — a small local model matches a large one on retrieval-limited questions; the larger model pulls ahead only on dense multi-firm synthesis. Either way, a model handed a chunk that lacks the fact will *fabricate* rather than abstain — which is exactly why the audit reads the cited chunk, not the answer's confidence.

One structural ceiling sits above all of this: **language.** On non-English filings accuracy falls off by language (English near-perfect, French partial, others weaker) — an embedding-and-extraction limit, not an isolation failure — so it is measured separately rather than hidden inside the score.

5.2 — What the mechanical score actually checks

A benchmark question carries `keywords` and an expected answer shape. A run marks a question **pass** on three checks, all mechanical:

1. every `keyword` appears in the answer,
2. the answer carries at least one citation,
3. the model didn't hedge with "no information available."

That's it. Notice what's **not** in the list: whether the answer is *correct*, whether it cites the *right firm*, whether the cited chunk actually *supports* the claim. The mechanical score is a presence test, not a truth test.

5.3 — The trap: a passing question can be wrong

Because the checks are presence tests, a question can pass all three while being substantively wrong. Real cases from the record:

- A KBC net-interest-income question **passed** — keywords present, citation present — while the answer was entirely about **ING**. Right keywords, wrong firm.
- A multi-firm question about JPMorgan / Bank of America / Citigroup **passed** while citing **Prudential Financial**. Right shape, wrong source.

The keyword check is a *signal*, not a guarantee of source correctness. This is the single most important thing to internalize about the harness: **mechanical pass ≠ quality.** We measured a

no-scaffold run at **8/8 mechanical and 1/7 audited** — a 100% mechanical score that was almost entirely wrong on inspection.

5.4 — The worst case: fabrication scores *higher* than honesty

The trap gets actively perverse when you compare models. On the same seven SEC questions, a fast **small model — which we'll leave unnamed** — ran 3× quicker and **fabricated**: an incoherent CET1, a revenue figure off by 10× ("$1,618 billion"), citations mapped to the wrong firms' filings, invented disclosure dates — while the standing model (gemma3:27b) correctly **abstained** where retrieval starved it. The mechanical score: **small model 7/7, gemma 6/7.** The keyword-and-citation gate *rewarded* the confident fabrication and *penalized* the honest abstention.

That's the strongest possible argument that the mechanical verdict, used alone, optimizes for exactly the wrong behavior.

5.5 — The fix to the verdict

We stopped trusting the pass-rate and moved the verdict to signals that catch fabrication:

- **Objective-%, not "pass-rate."** Score quality as the fraction of answers that are *objectively correct* — right content, right firm, supported by the cited chunk — out of every answer except the grading artifacts. In the four-state audit below, that's ✅ **Good ÷ (** ✅ **Good +** ⚠ **Partial +** ❌ **Miss)**; the 🔍 grading-artifact answers are excluded from the denominator because they're a measurement ceiling, not a model failure. We call it "objective %," deliberately not "honest %" — *honesty is a property of the measurement discipline, not a moral claim about the model.*
- **A four-state audit, not a binary.** Every audited question gets one of: ✅ **Good** (correct, right firm, substantive, cited), ⚠ **Partial** (mechanically passing but incomplete or wrong firm), ❌ **Miss** (retrieval or generation failure), 🔍 **Grading artifact** (mechanically failed but substantively *correct* — keyword brittleness, citation dropout, an accent mismatch). The grading-artifact bucket is the mirror image of §5.3: it's where the mechanical score *understates* quality, and it's what tells you to fix a keyword list rather than the engine.
- **An amber, entity-weighted score.** The weighted-sum grader leads with **entity coverage** (did the answer cite the firms it should) and demotes raw keyword presence to one signal among several — so confident-but-wrong-firm answers can't score green.

5.6 — The discipline that makes it real: verify the cited chunk

None of the above works on trust. A confident, well-cited, fluent answer can still be fabricated — the only thing that confirms a claim is **scrolling the cited chunk** and reading whether it actually says what the answer claims. Worked example: a CET1 answer that *looked* right was only

validated by pulling the exact Qdrant chunk and confirming the number and the column it came from. The rule — **verify-the-artifact**, or here, *verify-the-cited-chunk* — is why an audit is a read, not a re-run. A re-ingest that "completed successfully" is a claim; the chunk is the evidence.

5.7 — Running it, and the canonical settings

```
ais_bench --corpus sec_10k --top-k 10
```

writes a timestamped report to `benchmarks/<corpus>/reports/` in three formats (`.md` readable with answers, `.json` machine-readable, `.pdf` if `weasyprint` is installed). The harness reads the right parameters per corpus from metadata, so you rarely pass flags by hand. Canonical configuration: **α = 0.5, K = 10** for `sec_10k` and `esef_banks` ; **K = 5** for `demo` / `help` . Per-question `entity_filter` lives in the question YAML and is passed through to retrieval.

> ***Why K=10 for the multi-firm corpora — and why "effective K" is higher than it looks.*** *A cross-entity question ("compare X across these five firms") makes chunks from many firms compete for the same K retrieval slots — at a low K, a single verbose firm can crowd the others out and starve them of context (the zero-citation signature of §5.4). AIStudio raises the effective K for multi-entity queries via a per-entity retrieval quota, so each named firm gets its own slots rather than fighting for the global top-K. That's why* ***K=10 is the canonical depth for*** ***`sec_10k`*** ***and*** ***`esef_banks`*** *(vs. K=5 for the smaller* `demo` / `help` *corpora): it's the floor at which multi-firm questions stop starving. Raising K further trades precision for breadth and is not strictly monotonic per question — a larger candidate pool can push a borderline chunk below the reranker threshold — so K=10 is the tuned default, not a maximum. For your own corpora this is the main retrieval knob (see §1.4 and the annexes): raise K for breadth, lower for precision, and anchor cross-entity questions with explicit firm names.*
>
> ***The reference suite, and where it's defined.*** *There are two verbs.* ***`ais_bench --canonical`*** *reproduces the audited runs exactly — every run on its* ***pinned*** *model — which is what makes numbers comparable to the published ones.* ***`ais_bench --batch`*** *instead characterises your machine: it runs every set twice, once on the smallest model you have installed and once on the largest that fits, so you see the range your hardware actually delivers (narrow it with* `--model largest` , `--model smallest` , *or a model name).* `--batch-id <id>` *runs a single run from either. The suite — which corpora, scopes, question sets, and parameters each run uses — is defined in* ***`benchmarks/batch/bench_canonical.yaml`*** *; edit that file to change what the suite runs. Reports land in* `benchmarks/docs/` *(the published evidence suite). Think of* `ais_bench --batch` *as the release-time "regenerate the benchmark evidence" verb.*

The one-line takeaway for an operator: the green number is where you *start* reading, not where you stop. Run the bench for the signal, then audit the answers — read the cited chunks, check the firms — because the mechanical score's failure mode is to reward exactly the confident wrongness you most need to catch.

5.8 — Naming scopes so the benchmark binds them

> ***Tip — name scopes so the benchmark binds them automatically.*** *A scope file named* `<corpus>_<description>_scope.yaml` *lets you run* `ais_bench --corpus <corpus> --scope <description>` *without repeating the corpus name or a path —* `ais_bench` *resolves* `<corpus>_<description>_scope.yaml` *from the convention. For example, a file* `sec_10k_big_banks_scope.yaml` *is reached with* `ais_bench --corpus sec_10k --scope big_banks`*. The same naming works for the download scope (* `--scope` *), so one consistently-named file serves both. Keep your corpus's scope, questions, and metadata files together under* `data/corpora/<corpus>/`*.*

5.9 — Worked examples: four runs, read honestly

The principles above stay abstract until you watch them on real output. We ran four benchmarks — a **canonical suite** you can regenerate with one command (§5.7) — across the two shipped corpora, and read each the way §5.1–5.6 prescribe: start at the mechanical score, audit the answers, then state the **calibrated** read (the §5.5 metric — correct over correct-plus-partial-plus-miss, with architectural ceilings set aside). The four are chosen to expose two *orthogonal* frontiers: **A↔B** hold the corpus (US 10-Ks, English-clean) and vary question difficulty, isolating the **synthesis/temporal** ceiling; **C↔D** hold the question intent and vary the **language** surface of the corpus, isolating the cross-lingual ceiling. Together they show the §5.1 thesis from both sides — the score is a point in that space, not a verdict.

Run	Corpus · set	Mechanical	Calibrated (audited)	Frontier it maps
A	`sec_10k` · calibrated set	10/10	~9/10	dependable surface
B	`sec_10k_frontier` · hard set	10/10	~6.5–7	synthesis / temporal
C	`esef_blended` · 12 firms	8/12	~8–9/12	language (blended)

Run	Corpus · set	Mechanical	Calibrated (audited)	Frontier it maps
D	`esef_english` · 5 EN firms × 8 EN Q	6/8	~7–8/8	language (controlled)

```
ais_bench --corpus sec_10k                                        # Run A (calibrated set)
ais_bench --corpus sec_10k --questions frontier                   # Run B (hard set)
ais_bench --corpus esef_banks                                     # Run C (blended, 12 firms)
ais_bench --corpus esef_banks --scope lang_en --questions lang_en  # Run D (English-matched)
# or regenerate all four at once:
ais_bench --canonical      # reproduce the published numbers (pinned models)
ais_bench --batch          # characterise your machine (smallest installed + largest fitting
```

> *Run these (or `ais_bench --canonical`) and read each report in `benchmarks/docs/` alongside the analysis below — the point of §5 is reading a run honestly, not the headline number.*
>
> ***Don't benchmark a model that's too big for your Mac.*** *A benchmark run is just a series of queries, so the same memory-fit guard applies: point `ais_bench` at a model too large for your machine — `gemma3:27b` on a 24 GB Mac, which is the model the `--canonical` suite pins — and it would load and then hang, not error. AIStudio catches this: pass `--fit-policy` to say what should happen — `skip` (don't run it, logged loudly), `downshift` (auto-switch to the largest model that does fit), or `force` (run anyway, at your own risk). If you omit it on a run that won't fit, `ais_bench` **stops and offers what does fit**: several models fit → a numbered pick-list; exactly one fits → `(r)un it, (f)ree memory & restart, (Enter) cancel`; nothing fits → it stops and tells you how to free memory.*
>
> *On a memory-constrained box, prefer `ais_bench --batch` (which resolves the model to your machine automatically) over the pinned `--canonical` suite.*
>
> ***A caveat worth knowing, because it surprised us.*** *Fitting and sustaining are different questions. A model that loads happily can still run out of headroom part-way through a **multi-question sweep**, because each question's working memory accumulates. Measured on a 24 GB Mac: `llama3.1:8b` completed roughly seven of ten questions before the guard began declining the rest; `gemma3:12b` — which fits comfortably when idle — managed about one. Both are fine for asking one question at a time in the UI; neither reliably survives an unattended ten-question run on that machine. If a sweep's later questions come back instantly with no citations, that is the memory guard protecting you, not the model failing — free memory (`ollama stop <model>`) and re-run. Check any model's fit in the UI's model picker, in the `ais_start` summary, or with `curl localhost:8000/models`.*

Run A — the calibrated baseline. Mechanically 10/10; the audit calibrates it to ~9. The two soft spots are both **temporal label-binding** on multi-firm, multi-year questions — the answer is right in substance but binds a value to the wrong year-label — and critically, where coverage is thin the model **abstains and says so** rather than inventing (one question openly states the sources don't cover the asked 2024→2025 change). No fabrication; honest abstention. This is the *dependable surface* — the questions are framed (BIC) to the shapes the system reliably answers, with the hard table/temporal shapes deliberately deferred to B. **Direction:** the temporal-binding edge is what the verify-the-chunk discipline (§5.6) and the entity-KB work keep tightening; on the larger synthesis model these questions ground more firmly, so model choice is the near-term lever.

Run B — same corpus, the frontier set. Mechanically 10/10, calibrated to ~6.5–7, and the gap *is the point*. The three-firm, multi-year CET1-and-revenue question — a multi-column table query — passes mechanically with a confident, fully-formatted, citation-rich table that is **not reliable**: it reports the wrong year's ratio for one firm (right row, wrong column) and mixes capital bases across rows. This is the table-cell frontier (**Annex 4**) at full scale: maximally fluent, citation-rich, wrong in the cells — the clearest possible *mechanical pass ≠ correct*. A second loss is **temporal-span** (it compared two adjacent years instead of the asked multi-year span — distinct from the table problem, a synthesis fault). The third, a "dedicated AI governance committees" question, went the honest way: the filings disclose no body by that name (confirmed by direct retrieval — only general risk committees exist), so the model declined rather than invent — a correct answer to a question that presupposes something absent, not a defect. **Direction:** binding each number to its row, year, and basis is the active build — the structured-data and table-recognition modules (Annex 4); the frontier set exists to keep that boundary measured, not discovered live.

Run C — the blended-language corpus. Mechanically 8/12 across all twelve ESEF filers; calibrated to ~8–9, and here the mechanical score *understates* in places (some answers are correct but sparsely attributed — *read up*). The decisive pattern: **every soft spot clusters on a non-English firm.** The ambers and misses are Nordea, KBC, Erste — Swedish, Dutch, German filings where an English question retrieves against non-English text (Erste is the multilingual-tokenization case of **Annex 3**). There is **zero fabrication** — the failure mode is low-density or partial retrieval, never a confident wrong answer. This run *contains* the language ceiling rather than isolating it. **Direction:** the cross-lingual retrieval work of Annex 3 — terminology bridging, filing-language tagging, a larger model on the dense non-English passages.

Run D — the same corpus with language controlled. Mechanically 6/8, calibrated to ~7–8 and effectively clean: the five genuinely-English filers (scope `lang_en`) against the eight English-appropriate questions (`--questions lang_en`). Every question that was an amber in C *because of a non-English firm* is simply absent here — so D is the apples-to-apples "what does the system do when language is controlled" number, and it comes back essentially clean. The two ambers are **keyword artifacts only** (a brittle "regulatory" literal, plus one low-density-but-

correct answer), not retrieval failures. **C↔D is the entire language story in one pair: 8/12 blended → ~7–8/8 on the controlled English cut. Direction:** the residual ambers are grading-token artifacts, not capability — they point at the scoring rubric, not the retrieval.

Overall assessment. Across both corpora the system is solid where the audits say it is: firm isolation on well-tagged corpora, narrative and cross-firm qualitative synthesis, and single-fact English lookups come back grounded and correctly cited. The frontiers are equally clear and equally stated — multi-firm **table-cell extraction** (B's capital table, Annex 4), **multi-year temporal synthesis** (B's span question), and **non-English retrieval** (C's Nordea/KBC/Erste, Annex 3). The two axes are orthogonal: A↔B isolates synthesis on English-clean US filings; C↔D isolates language on the European corpus — and the constant across all four is **graceful failure: zero fabrication, honest abstention where coverage is thin.** None of this hides inside a green number; each frontier was found by reading the answers, which is the method. And the direction of travel is named throughout: structured-data and table-recognition for the table frontier, the multilingual work of Annex 3 for non-English filings, a richer model for dense synthesis. The benchmark's job is not to award a grade — it is to keep that map of solid ground and frontier honest, current, and visible, run after run. The companion case study (`ART - AIS - AIS vs Google vs Reality`) dramatizes the same citation-grounding discipline narratively.

5.10 — Verify the bench ran what you think it ran

Every reading above assumes the run actually executed the questions you meant, against the corpus you meant, with the filter you meant. That assumption fails silently and the failure looks exactly like a model regression — a cliff in the score with no code change behind it. Before you trust *any* number (and certainly before you cite one), run this five-question pre-flight. Each maps to a real failure mode that produces a believable-but-false score.

1. **Did the run load the questions on disk now?** The harness reads a default question file unless you pass `--questions`. If your repo checkout and your working tree disagree (a gitignored questions file that never traveled, a stale copy in a clone), the run grades a *different* set than the one you're holding. Check the run's recorded `questions_sha8` against the hash of the file you intend: `shasum -a 256 <questions.yaml> | cut -c1-8`. If they differ, the run is void — re-run with `--questions <path>` to pin it.
2. **Is the file on disk the set you think it is?** Hash-matching a stale file just confirms you ran the stale file. Eyeball the question `id`s and confirm they're the curated set (for sec_10k that's the BIC default; the harder `June_2026` set is a separate file). A run against last month's questions is not comparable to this month's.
3. **Does every `entity_filter` token match a real corpus filename?** The filter binds on a filename substring, not on the LEI — so a token that drifted from the ingested slug (`Standard_Chartered_PLC` when the file is `StandardChartered_Bank`) silently retrieves **nothing**, and the question scores as a confident miss that is really a dead filter. Cross-

check every token against `ls data/corpora/<corpus>/uploads/` before trusting a low score on an entity-filtered question.

4. **Did any question come back with zero citations?** Zero citations on a filtered question is the signature of #3 (or of genuine retrieval starvation at low K — see Run C). Either way it is not a model-quality signal; it is a plumbing signal. Separate the zero-citation answers out and diagnose them before they drag the objective read down.
5. **Right model, right corpus?** The report filename and config record both. A "regression" is often a small model where you meant a large one, or the default corpus where you meant a scope. Confirm `--model` and `--corpus` / `--scope` in the run's recorded config match your intent.

The discipline here is the same as §5.6, pushed one level earlier: §5.6 verifies the *answer* against its chunk; §5.10 verifies the *run* against your intent. A clean four-state audit on a run that loaded the wrong file is a rigorous reading of the wrong thing.

If you have worked through all of this, you are equipped to experiment freely — swap models, reshape questions, build your own corpora, and read the results honestly. AIStudio keeps growing: we are sharpening **query understanding** (reading the intent behind a question), building modules to ingest other kinds of **structured data**, improving **data recognition inside tables** (the active frontier — see **Annex 4**), and bringing **multimodal** inputs into the same pipeline. Happy Building :-)

Annexes

The modules walk the happy path. What follows is deliberately a mix of three different kinds of thing, and it helps to know which one you're reading at any moment:

- ***How the system is actually structured*** *— the weeds. How the shipped corpora were really built, where each piece lives, and how the parts fit together. This is not an architecture description. For the general picture — the components, the objects, and especially the data and how it flows — read the companion note AIStudio — Elements of an Architecture.*
- ***Lessons learned building it*** *— findings from the problem space itself, including the parts that fight back: pulling numbers out of dense tables, and handling filings that aren't in English.*

- ***The shape of how we evaluate*** *— the broad contour of the method we use to measure results continuously. Getting this right was a journey, not a one-time setup, and the annexes show the turns.*

So these annexes are really a collection of findings and recipes — the sort of material that would normally live in a wider series of technical notes written for engineers (and we may yet spin them out of what has become a fairly bloated tutorial). But we think they're also for anyone curious about how a messy technical problem can be approached systematically. Enjoy!

Annex 1 — How we create reference data for entities

This is the meaty annex, because entity resolution is where reality bites. It's a pipeline narrative, not a command dump: a membership list becomes verified filings, which become a reviewed entity KB, which becomes the chunk labels you saw in Module 2.

A1.1 — The membership list (the scope)

A downloaded corpus's roster is **not** hardcoded — it lives in a **scope file** at the corpus root, `data/corpora/<corpus>/<corpus>_full_scope.yaml`: one row per firm, keyed by a readable `label` and carrying the identifiers the downloader needs. The access key differs by source, but the **identity key is always the LEI**:

- `sec_10k` → rows of `{label, cik, ticker, lei}` — downloaded from **SEC EDGAR** by CIK.
- `esef_banks` → rows keyed by **LEI** — downloaded from **filings.xbrl.org**.

The two corpora that ship in the box are built differently and carry no such roster: **demo** is ingested from documents bundled with AIStudio, and **help** is generated from AIStudio's own documentation. The scope file is how a *downloaded* corpus declares which firms it contains.

Where each source is reached — endpoint, access key, rate limits — is kept as data, not baked into code (`data/knowledge_sources/gleif/gleif_metadata.yaml` carries the base URL and endpoints for GLEIF; EDGAR and filings.xbrl.org are configured the same way). Adding a regulator is configuration, not a rewrite. To add or remove a firm, edit the scope file — never the downloader.

A1.2 — Pulling the filings

The downloader reads a **scope** — a membership list of `{label, cik|ticker}` — and fetches one firm at a time (default: the 5 most recent annual filings), writing into `data/corpora/<corpus>/uploads/`. Module 2's bare `ais_download_sec_10k` was scope-driven all along: it

reads the default `data/corpora/sec_10k/sec_10k_full_scope.yaml`. You can also target a single firm, or supply your own list:

```
ais_download_sec_10k                              # default scope (the curated portfolio)
ais_download_sec_10k --cik 0000886982             # one firm, by CIK
ais_download_sec_10k --tkr BNY --years 5          # one firm, by ticker
ais_download_sec_10k --scope my_firms.yaml        # bring your own list (same schema as the
ais_download_esef --scope lang_en                 # ESEF: a language-segmented subset
```

Bring your own list (Beat 2). The default scope is just a file. Copy `sec_10k_full_scope.yaml`, keep the schema (`entities: [{label, cik|ticker}]`), list the firms you want, and run `--scope <your.yaml>`. The "you were lucky" set of Module 2 was lucky precisely because someone already curated that file — every firm in it posts clean iXBRL with the entity tag. Your own list won't be pre-vetted, which is what the next part is about.

Verify at download (Beat 1). With the curated scope every filing carries the tag ingest needs. Off the curated path that's not guaranteed — so the downloader **verifies each filing as it lands**, reading the same `dei:EntityRegistrantName` tag the ingest pipeline's primary strategy reads, and reports ✅/❌ per filing. Worked example — try a firm whose older filings predate the tag:

```
ais_download_sec_10k --tkr BNY --years 5
```

BNY Mellon's older 10-Ks carry `dei:AuditorName` but **not** `dei:EntityRegistrantName` (the same class as Wells Fargo). So verify comes back all-❌ and the downloader coaches:

```
❌
  ...: none of the downloaded filings (2021, 2022, 2023, 2024, 2025) carry an iXBRL
  entity tag (dei:EntityRegistrantName), so ingest can't auto-recognize them.
  You can override:
    ais_download_sec_10k --tkr BNY --years 5 --force_name "<name to use>" [--force_year <Y
  But it's worth finding out *why* the filing lacks the tag first — see Tutorial Annex 1.
```

`--force_name` / `--force_year` let you assert the identity the tag didn't provide. This is the **download-time twin of the worksheet's name-less lever** (A1.4 case 3): same fix — the operator supplies the name — done early, at fetch, instead of later, at the worksheet. The assertion is recorded to a sidecar (`sec_10k_entity_overrides.yaml`) so it survives to ingest. But the coaching is sincere: a missing mandatory tag usually means you've got the wrong document or a pre-iXBRL filing, and forcing a name papers over that rather than fixing it.

A1.3 — Recognizing entities (scan → guess → worksheet)

Running `ais_import_entity_kb --corpus <corpus> --apply` does three things:

1. **Scan** every filing in `uploads/` and read its self-reported iXBRL entity name (`dei:EntityRegistrantName`).
2. **Best-guess** each against GLEIF.
3. If everything resolves cleanly, **apply** — write the entity KB. If any row needs a human, the **`full_scope` itself is the worksheet**: there's no separate file — you edit `data/corpora/<corpus>/<corpus>_full_scope.yaml` in place and re-run.

The doctrine, stated plainly: **the machine guess is a starting point; the human-verified LEI is the input; the scope is the trust boundary.** A name search can return a confident, well-formed, *wrong* legal entity — and a clean ingest of the wrong entity is invisible to every downstream check. The scope exists so a human confirms the identity once, before any of that data is trusted.

The scope is an open schema. Every non-`label` field names its own provenance, and the **bare** field is the user-authoritative one that **wins**:

```
- label: BlackRock                # the handle (user)
  cik: 0002012383                 # bare — the download key (user/seed)
  ticker: BLK                     # bare (user)
  lei: REPLACE_WITH_VERIFIED_VALUE_IF_AVAILABLE       # bare — YOUR verified LEI; WINS. Sent
  xbrl_name: REPLACE_WITH_VERIFIED_VALUE_IF_AVAILABLE # bare — user name override; sentinel
  sec_xbrl_name: BlackRock, Inc.   # the filing's self-reported tag (source; populated by sc
  gleif_lei: 529900VBK42Y5HHRMD23  # GLEIF's confirmed LEI (source)
  files: [CIK_0002012383_10K_2025-02-25.htm]
```

The rule is `<provenance>_<data_type>` for a source's value (`gleif_`, `sec_`, `wikidata_`) and a **bare `<data_type>`** for the human value. Resolution reads **bare → source → empty**: a verified bare `lei` overrides the machine-guessed `gleif_lei`; a bare `xbrl_name` overrides the filing's `sec_xbrl_name`. The bare field *is* the canonical — there is no separate `canonical` field. Unknown bare fields ship as `REPLACE_WITH_VERIFIED_VALUE_IF_AVAILABLE`, a fill-me marker the resolver treats as empty. This is what makes the schema **open**: a new source is a new prefixed field, and a human correction is always one bare field away — neither touches code.

A1.4 — When reality bites: the four levers

Every row the scan can't resolve falls into one of four cases. Each has one fix in the worksheet:

1. **Resolved-but-verify.** The guess looks right; confirm it. *Example: Jefferies — resolves, but you confirm the registrant collapse is the operating company, not a holding shell.*

2. **Name-resolution failure → fill `lei`.** The name search returned the wrong entity (a subsidiary, a depositary receipt, a namesake). You paste the correct 20-character LEI into `lei` and re-run; the row is rewritten deterministically by LEI. *Examples: AllianceBernstein and Wells Fargo (US); Société Générale (→ US sub), Crédit Agricole (→ Brazilian sub), BBVA (→ Santander ADR) on the European set.*
3. **Name-extraction failure / name-less → author the bare `xbrl_name`.** The filing's entity tag is missing or mangled (e.g. inline-markup whitespace collapse → "Erste GroupBankAG"), so there's no name to resolve. You author the entity name in the bare `xbrl_name` field. *Example: BNY Mellon and the whitespace-collapse class.* This is the same assertion `--force_name` makes at download time (A1.2) — the scope is the later, canonical place to make it.
4. **Wrong/missing entity at ingest → the `expected_xbrl_name` guard.** A filing whose self-reported entity doesn't match the target's `expected_xbrl_name` is **skipped unless `--force`** — so a mis-mapped filing can't silently ingest under the wrong firm. *Example: the CBOE filings that were actually Larimar Therapeutics' 10-K under a wrong CIK — a clean ingest of the wrong company, caught only by this guard.*

The first time through, you fill `lei` for the failures, re-run, and the table goes clean. From then on the edits persist across re-runs — you review once.

A1.4b — The scope's open/provenance field convention — and why LEI is the shared key

The `full_scope` file is the single source of truth for corpus membership. Each entry is keyed by `label`; every other field encodes two dimensions:

```
<provenance>_<data_type>   — a source's reported value (sec, gleif, wikidata, esef, ais)
<data_type>                — bare = the user/by-hand value, which is AUTHORITATIVE and wins
```

The **bare field IS the canonical** — there is no separate `canonical` field. A source's guess lands in its prefixed field; you fill the bare field only to override a bad guess. **Resolution rule:** bare (user) → golden source(s) → empty.

data_type	bare (user, wins)	source field(s)
`lei`	`lei` — operator-typed LEI; **the shared key, used thereafter**	`gleif_lei`
name	`xbrl_name` — user-authored name	`sec_xbrl_name` / `esef_xbrl_name`, `gleif_name`

data_type	bare (user, wins)	source field(s)
`cik` / `ticker`	`cik` / `ticker`	`sec_cik` / `sec_ticker`
`jurisdiction` / `filing_language`	bare	`sec_*` / `esef_*`
`aliases`	`aliases`	`gleif_aliases` ∪ `wikidata_aliases` → `combined_aliases`
`expected_xbrl_name`	seed — download-verify guard (fetched filing must match)	—
files / last_updated	tool-managed (written by download/ingest; not hand-edited)	—

For LEI specifically: `gleif_lei` holds what GLEIF confirmed; if you find a better one, fill the bare `lei` and it wins. There is **no `lei_corrected`** — vestigial, replaced by this convention.

This is why the 2026-06-16 ESEF build succeeded despite five firms filing under names we didn't expect — Nordeakoncernen, Erste Group BankAG, Barclays Bank PLC, StandardChartered Bank, Skandinaviska Enskilda Banken AB (publ). The LEI resolved each identity correctly regardless of what the filer typed. For European entities, **supply the LEI — the rest resolves from there.**

> ***Implementation status.*** *This open/provenance convention is **live**: the ESEF corpus scopes ship in this form and the resolver reads them directly — Module 3 was built end-to-end on it. The SEC scope is the remaining laggard: its rows still carry the older names and are being migrated to match.*

A1.5 — Correcting a LEI: a worked example

When a row resolves by name-search, the review prints it as `(name-search, not verified LEI)` — the LEI is a machine guess sitting in `gleif_lei`, and the bare `lei` is still the sentinel. Promoting it to verified is one edit. BlackRock is the live case: added on demand, it name-searched to `529900VBK42Y5HHRMD23` — the post-2024-reorg successor (previous legal name

BlackRock Funding, Inc., confirmed at search.gleif.org). The guess was right, but it's still only a guess until a human says so.

1. Open the scope in an editor: `bash open -e ~/Developer/AIStudio/data/corpora/sec_10k/sec_10k_full_scope.yaml`
2. Find the BlackRock row and set the **bare** **`lei`** to the value you verified at GLEIF, replacing the sentinel. Note the row's `label` depends on *how you downloaded the firm*: it's `CIK_0002012383` if you fetched by `--cik` (the downloader's synthetic label), or `BlackRock, Inc.` if you used `--tkr BLK` (the resolved name). Grep by CIK or name rather than assuming the label: `grep -n "2012383\|BlackRock" <scope.yaml>`. And if you pulled a multi-year history, BlackRock is **two rows** — the successor (`cik: 0002012383` , recent years) and the predecessor (`cik: 0001364742` , older years, pre-reorg) — set the bare `lei` on **both**, since they share one GLEIF LEI: `yaml lei: 529900VBK42Y5HHRMD23`
3. Re-resolve and rebuild — **no re-ingest** (chunks are untouched): `bash ais_import_entity_kb --corpus sec_10k # now resolves BY your verified LEI ais_import_entity_kb --corpus sec_10k --apply # rebuild the KB ais_start # reload the cached KB`

The review line flips from `(name-search, not verified LEI)` to a clean resolve: the bare `lei` you typed now **wins** over `gleif_lei`, so the row is verified by a human, deterministically. That precedence — the bare user value over the machine guess — is the whole point of the open schema.

A1.6 — Apply, and what you get

When the table is clean, `--apply` writes `data/knowledge_sources/gleif/gleif_<corpus>_full_entities.yaml` (schema 1.1):

```
schema_version: '1.1'
source_id: gleif
corpus: <corpus>
scope_id: full
entity_count: <N>
entities:
  - canonical: <GLEIF canonical name>
    scope_name: <the natural query form — what users type>
    lei: <20-char LEI>
    aliases: [<short names, tickers, case variants>]
    wikidata_label: <optional>
    wikidata_short_name: <optional>
    wikidata_tickers: [<optional>]
```

Each firm is **one entry in the `entities:` list**, identified by its **LEI**. At ingestion the pipeline reads the chunk's self-reported entity, matches it against this list, and prefixes the chunk `[Document: <canonical> | <scope_name> | <ticker> FY<year>]` — so the firm and year travel inside the chunk text. The LEI stays in the KB as the **identity key**; it is deliberately **not** in the prefix (it isn't a retrieval token and would only add noise to keyword matching). `scope_name` is the human-facing name the prefix and queries use; `aliases` widens lexical recall.

Practical order for adding a US firm to a scope: find it on **EDGAR** to confirm it files a domestic 10-K and grab its **CIK**; cross-check the **ticker** in `company_tickers.json`; look up the **LEI** on **GLEIF** for the `lei` field (used by entity resolution and the benchmark scope). For a European bank, the **LEI** is the primary key — start at **GLEIF**, then confirm filings exist on **filings.xbrl.org**.

> *Identifiers are reference data: verify them at the source, don't trust a guess. A wrong CIK silently downloads the wrong company's filing — and EDGAR is the only authority that settles it.*

A1.7 — The European parallel

Module 3 runs this exact flow with the LEI as the *download* key (filings.xbrl.org filters by LEI prefix). The headline lesson lives here: on the European set a name search confidently mismatched roughly half the firms — the strongest possible argument for LEI-is-input. The worked `lei` corrections for the European banks are the canonical example of the second lever above.

Annex 2 — How we create reference data for the glossary

Deliberately short — it's the contrast to Annex 1. The glossary is the *symmetric, simple* reference layer: no API to fight, no entity to disambiguate, no review gate.

A2.1 — What it is

A glossary maps a domain term to an expansion string. It is applied **at query time only** — chunks are never rewritten. This is the asymmetry that makes it the opposite of the entity KB (which is baked into chunks at ingestion and used as a retrieval filter). Understanding the contrast is the teaching point: two reference layers, one human-gated and build-time, one curated and query-time.

A2.2 — The curated source

The bank corpora use **`bis_basel`** — Basel III/IV regulatory vocabulary, a static, hand-curated source. Because it's deterministic, there is **no review gate and one pass** — nothing to resolve against a flaky registry, nothing for a human to confirm. (Contrast: the BIS SCO95 source page renders via JavaScript and isn't scrapeable, which is exactly why it's curated rather than fetched.)

```
ais_import_glossary_kb --source bis_basel
```

writes `data/knowledge_sources/bis_basel/bis_basel_<corpus>_<scope>_glossary.yaml`. Each entry maps a term to its full form and a keyword expansion string:

```
glossary:
  - term: CET1
    full_form: Common Equity Tier 1
    expansion: CET1 Common Equity Tier 1 capital ratio regulatory capital
  - term: RWA
    full_form: Risk-Weighted Assets
    expansion: RWA risk-weighted assets capital requirements standardized approach
```

A2.3 — How it's used

At query time, AIStudio looks up each glossary term in your question and widens the keyword (BM25) search to include its expansion — so *"CET1"* reaches *"Common Equity Tier 1 capital ratio regulatory capital"* and a question phrased either way finds a passage phrased the other way.

A2.4 — Extending it

`ais_import_glossary_kb` takes `--source`, `--corpus`, and `--scope`. Additional sources (`nist_ai_rmf`, `esrs`) are declared but not yet built. A new glossary source is a new curated seed plus one import run — no worksheet, no review.

The spine across both annexes: two reference layers — **entity** (build-time tag + query filter, human-gated, asymmetric) and **glossary** (query-time expansion, curated, symmetric). Annex 1 earns its length because the data fights back; Annex 2 is short because it doesn't.

Annex 3 — When the portfolio isn't in English

A US-only corpus hides a problem that the European set surfaces on the first query: **retrieval quality is not language-neutral.** Everything in Modules 2–3 — embeddings, BM25, the glossary, the relevance threshold — was tuned against English text, and each one degrades differently when the filing isn't in English. This annex is the honest account of where the language ceiling is, what we can do about it, and what we can't.

A3.1 — Why a European portfolio is harder than a US one

Two things change at once when you cross the Atlantic, and they compound:

- **The filings aren't all in English.** ESEF filers report in their primary language — French, German, Dutch, Norwegian — sometimes with an English version, sometimes not. A question in English against a filing written in French is a cross-language retrieval problem the US corpus never poses.
- **The legal structure is messier.** European groups carry more subsidiaries, depositary receipts, and namesakes — which is why entity resolution (Annex 1) fails far more often here. A name search for these banks confidently mismatched roughly half the set: Société Générale → a US subsidiary, Crédit Agricole → a Brazilian subsidiary, BBVA → a Santander depositary receipt. That's an *identity* problem (Annex 1's `lei` fix), but it has a *language* root — the jurisdiction default and the multilingual registry are what make the guess wrong.

The observable result: the European corpus runs materially below the US one on objective quality even after the entity work — the gap is the language ceiling. We deliberately don't reduce it to a single percentage: the 17-firm set isn't a random sample, so a headline number would imply more precision than the observation supports.

A3.2 — The three places language degrades retrieval

1. Terminology mismatch — the glossary is English-anchored. Ask about *"CET1"* and a French filing may only ever say *"fonds propres de base de catégorie 1."* The BIS Basel glossary (Annex 2) expands the *English* acronym to its English full form — it does **not** translate. So the glossary closes the acronym↔full-form gap *within* English but does nothing for the English↔French gap. Cross-language terminology is an open seam, not a solved one.

2. Mislabeled `filing_language`. A corpus row that claims a filing is English when it isn't poisons everything downstream — scope filtering, any language-conditioned expansion, the operator's mental model of the set. Worked example: **DNB** was carried as `filing_language: en` when the filings are Norwegian; the fix is to correct the label and tighten the `lang_en` scope

(12 EN-primary firms, not all 17). The lesson generalizes: the language tag is reference data like the LEI — verify it, don't trust the downloaded default.

3. Accent and diacritic mismatch. A query token without diacritics (`Credit Agricole`) and a filing token with them (`Crédit Agricole`) are different strings to a naive matcher, and the accented form can score **below the relevance threshold** and drop out before the model ever sees it. Worked example: **Crédit Agricole**'s French terminology fell under `min_score` on an early run — a retrieval miss caused purely by orthography. The mitigation is accent normalization at query and index time (folded into the harness), but normalization is a patch on a tokenizer that wasn't built for the problem.

A3.3 — The levers we actually have

- **`lang_*` scope segmentation.** The scope taxonomy (`lang_en`, `lang_fr`, `lang_en_fr`, `lang_not_en`) lets you benchmark and tune a language-coherent subset instead of averaging across a mixed set, so a French-filing failure doesn't hide inside an English-majority score. `esef_banks_lang_en_scope.yaml` (the EN-primary subset) + its basic question set are the canonical instance.
- **A bigger model buys multilingual headroom.** On the same European questions, the larger synthesis model showed better citation discipline and visibly better multilingual handling than the small one — at a latency cost. This is one of the few language levers that helps across the board rather than patching one failure mode.
- **α tuning is not the fix — entity isolation is.** Pushing the retrieval mix toward keyword (α≈0.75) fixed one firm's dominance (ING) but immediately introduced another's (Santander) — squeezing the balloon. The durable fix for "one firm's chunks crowd out another's" is the per-firm retrieval quota and the entity filter (Module 2's chunk labels), not a global α knob. α is a per-query convenience, not a corpus-level cure.

A3.4 — The honest limits

This is a **frontier, not a closed problem.** Cross-language terminology has no glossary bridge yet; non-Latin and heavily-accented text still stresses the tokenizer; and a multilingual corpus's objective-% sits structurally below an English one until those are addressed. The right operator posture is the same as everywhere else in AIStudio: segment by language so the failures are visible, verify the language labels like any other reference data, and read the cited chunks (Module 5) rather than trusting a score that English tuning inflated.

Annex 4 — The Table Problem (Ahhh… tables…)

Everything to this point assumed the answer lives in a *sentence*. Financial filings break that assumption constantly: the number you want sits in a cell, and a cell only means something through its **column header** (the period — *FY2025* vs *FY2024*) and its **row header** (the variant — *Standardized* vs *Advanced*). RAG chunks a document into ~1,200-character windows; when a chunk boundary falls between the header band and the data row — or when the model reads across a wide row to the wrong column — the cell arrives stripped of the headers that gave it meaning. The model then answers fluently and **confidently wrong**. This annex is one worked case where the failure and the success sit side by side, because the contrast *is* the teaching point.

A4.1 — The query that exposes it

Ask the SEC corpus a cross-firm numeric question:

> *"Compare the CET1 ratios for JPMorgan and Citigroup."*

Grounded in each firm's FY2025 10-K, AIStudio returns JPMorgan **≈ 15.7%** and Citigroup **13.2%**, both labeled *as of December 31, 2025*, each citing the correct firm's filing. Entity isolation worked — right firms, right documents, no cross-contamination. **But one of those two numbers is exactly right and the other is a full year wrong** — and nothing in the answer tells you which.

A4.2 — Why AIStudio's reading of Citigroup's report is correct and its reading of JPMorgan's is not: prose vs. grid

The difference is *how each filing states the number*, not which firm the system prefers.

- **Citigroup states it in a sentence.** The Citi 10-K's Capital section says, in prose, that the CET1 ratio was **13.2%** as of December 31, 2025, compared with **13.6%** a year earlier, on the Standardized approach. A sentence keeps the value bonded to its period and its basis — so it survives chunking intact. AIStudio reproduced all of it (the 13.2%, the 13.6% prior year, the Standardized-is-binding point) faithfully.
- **JPMorgan states it in a multi-year column table.** JPM's CET1 row reads, across columns, **14.6% (2025) · 15.7% (2024) · …**. The true *as-of-December-31-2025* figure is **14.6%**. AIStudio returned **15.7%** — the **2024 column** — and labeled it 2025. Right row, wrong column. (Confirmed against the primary filing and JPM's own February-2026 disclosure of a 14.6% current Standardized ratio.)

That is column-header detachment in its purest form: the prose figure survives, the grid figure loses the year and lands one column off.

A4.3 — The second loss: the row header (and the tell it leaves)

The same mechanism corrupts the *variant* (the row header) — and it leaves a tell you can catch without ever opening the source. Both firms came back with an "Advanced" figure that is **structurally impossible**:

- AIStudio gave JPM an Advanced ratio (≈15.8%) **higher** than its Standardized — but JPM's 10-K states that as of December 31, 2025 the *Advanced* approach became the **more binding** (i.e., lower) one. Advanced above Standardized cannot be right.
- AIStudio gave Citi an Advanced ratio (≈11.9%) **below** its Standardized 13.2% — yet the same answer correctly said Citi's *binding* ratio is the Standardized one, which is only true if Advanced is the *higher* (non-binding) number. An Advanced below the binding Standardized is self-contradictory.

Catching this needs no source at all: each answer **contradicts itself**. That is the operator's cheapest table-misbind detector — when a derived comparison violates a rule the answer itself states (binding = the lower of the two), suspect a row/column detachment, not a real datum.

A4.4 — Why it's hard, and what AIStudio does about it

A naïve text extractor flattens a table to a stream of numbers and the headers dissolve. AIStudio's ingestion runs a table-extraction normalizer (`loaders.py` / `chunking.py`) that detects grids and serializes them to header-bonded markdown rows, so a clean single-header table survives as `| CET1 capital ratio | 13.1 | % | … |` with the label attached — that closes the common case. The open frontier — the "hard 10%" — is exactly what the JPM example hits: **stacked / multi-level column headers** (a year band sitting *over* a Standardized/Advanced band) and **chunk boundaries that sever the header band from the data rows**. Composing multi-level headers into each data cell (`label (Standardized, FY2025): value`) is the header-binding work tracked as the table-understanding item; until it lands, multi-year multi-basis tables are the residual risk.

A4.5 — The operator's defenses, today

- **Name the column in the query.** Asking for *"JPMorgan's consolidated holding-company Standardized CET1 ratio as of fiscal year-end 2025"* hands the retriever and the model the column and row keys explicitly — in the record, re-asking a misread CET1 question *with the column named* returned the correct cell from the very same chunk. The data was always there; the query supplied the missing header.

- **Verify the cited chunk — including the column.** This is Module 5's discipline (§5.6) applied to tables: scroll the cited chunk and confirm not just that the number appears, but that it sits under the period and basis the answer claims. A table answer is verified at the *cell*, not the page.
- **Trust prose over grids, and cross-check the period.** A figure stated in a sentence can be weighted; a figure lifted from a table should be treated as needing confirmation. A two-firm comparison where one number is prose-sourced and the other grid-sourced — this exact case — is the highest-risk shape there is.

A4.6 — The honest limit

Prose-stated numbers extract faithfully; **table-grid numbers are a frontier — not a solved problem, at least not completely by AIStudio, but actively worked on** as we evaluate several techniques and libraries to address the challenge. Single-header grids are handled today; stacked-header multi-year tables are not yet. The posture is the one the whole tutorial keeps returning to: read the cited chunk, distrust a confident derived comparison that contradicts itself, and treat a table figure as a claim to verify rather than a fact to repeat. The system's job is to make the misbind *visible* — the self-contradiction in A4.3 is that visibility — and closing it for good is the header-binding serializer's job.

Annex 5 — Under the Hood: Where Data Lives

The modules keep file paths out of the way on purpose. This annex is the map — where AIStudio stores each piece on disk, so you can find, back up, or inspect it. Everything sits under the AIStudio install directory.

What	Where	Notes
Source documents	`data/corpora/<corpus>/uploads/`	The ingested filings, or your own uploaded files.
Membership manifest	`data/corpora/<corpus>/<corpus>_full_scope.yaml`	The `{label, cik|lei}` rows driving download + entity resolution (**Annex 1**).
	`data/corpora/<corpus>/<corpus>_corpus_metadata.yaml`	Query defaults + ingest counts. Edit

What	Where	Notes
Corpus settings + stats		via the UI **Edit** modal.
Entity knowledge base	`data/knowledge_sources/gleif/<corpus>_full_entities.yaml`	Canonical name + LEI + aliases per firm (**Annex 1**).
Glossary knowledge base	`data/knowledge_sources/bis_basel/bis_basel_<corpus>_<scope>_glossary.yaml`	Term → full form → keyword expansion (**Annex 2**).
Source metadata	`data/knowledge_sources/gleif/gleif_metadata.yaml`	Endpoints, base URL, rate limits — kept as data, not code.
Benchmark questions	`benchmarks/<corpus>/<corpus>_questions.yaml`	The benchmark suite (**Module 5**); reports in `reports/`.
The vector index	Qdrant collection `aistudio_<corpus>`	The embedded chunks; rebuilt on every ingest (`--force` wipes first).

The shape of it: everything *about* one corpus lives under `data/corpora/<corpus>/`; the *shared* reference data — the entity and glossary knowledge bases, and source metadata — lives under `data/knowledge_sources/`; and the vector index lives in Qdrant. The source documents and the two YAML knowledge bases are the human-editable inputs; the metadata file's ingest stats and the Qdrant collection are machine-written outputs.

Reference — Commands

Command	What it does
`ais_start`	Start all services and open the UI
`ais_stop`	Stop all services

Command	What it does
`ais_download_sec_10k`	Download SEC 10-K filings (reads the corpus scope file)
`ais_download_esef`	Download European ESEF reports (reads the scope file, by LEI)
`ais_import_entity_kb --corpus <name> --apply`	Build a corpus's entity KB (resolves firms by verified LEI)
`ais_import_glossary_kb --source bis_basel`	Build/refresh the domain glossary (ships ready-built)
`ais_ingest_sec_10k` / `ais_ingest_esef`	Ingest a corpus (~30 min); applies chunk enrichment. Incremental by default; `--files <patterns>` ingests only matching files (comma-separated substrings/regexes, OR-matched); `--force` rebuilds
`ais_bench --corpus <name>`	Run the benchmark suite for a corpus
`ais_log` · `ais_help` · `ais_install`	Tail log · command reference · install/update

Reference — Sources & Lookups

When you build your own scope (Module 4, Annex 1) you'll need to look up identifiers by hand. These are the authoritative, free sources for each:

You need	Where to look it up	Source
CIK (US filer ID) and the filings themselves	Company/CIK lookup — `https://www.sec.gov/search-filings/cik-lookup` · full-text search — `https://www.sec.gov/edgar/search/`	**SEC EDGAR** (U.S. Securities and Exchange Commission), `https://www.sec.gov/edgar` ; data API `https://data.sec.gov`
Ticker → CIK (the map AIStudio resolves `--tkr` against)	`https://www.sec.gov/files/company_tickers.json`	**SEC EDGAR**

You need	Where to look it up	Source
LEI (the 20-char legal entity ID — the entity-KB input)	LEI search — `https://search.gleif.org` (US alt: OFR finder `https://www.financialresearch.gov/data/legal-entity-identifier/find-lei/`)	**GLEIF** (Global Legal Entity Identifier Foundation), `https://www.gleif.org`; API `https://api.gleif.org/api/v1`
European (ESEF) filings, by LEI	`https://filings.xbrl.org`	**filings.xbrl.org** (operated by XBRL International)
Basel terms behind the glossary	Basel Framework — `https://www.bis.org/basel_framework/`	**BIS** (Bank for International Settlements)
The **ESEF mandate** itself (why European filings are iXBRL)	`https://www.esma.europa.eu`	**ESMA** (European Securities and Markets Authority)